

Linear Regression Assignment — Study Notes

These notes summarize the ML Lab Assignment 2 and explain what is required, what concepts are used, and how the different sections fit together.

1. Big Picture

Goal: predict **median_house_value** from the housing features. The assignment requires Linear Regression using NumPy with the Normal Equation and Batch Gradient Descent, followed by comparison with scikit-learn.

Overall flow: train.csv → preprocessing → train models → predict test.csv → calculate metrics → create visualizations → compare results.

2. Section A — Data Preprocessing

Use **train.csv** for training and **test.csv** for validation. Separate the input features X from the target y . The target in this dataset is **median_house_value**.

Required steps:

- Separate X (features) and y (target).
- Standardize every feature to zero mean and unit variance.
- Calculate mean and standard deviation from the training data.
- Use the same training mean and standard deviation to scale the test data.
- Add a column of ones to X for the intercept/bias when using the NumPy implementations.

Standardization formula:

$$X_{\text{scaled}} = (X - \mu) / \sigma$$

Why scaling? The features have very different numerical ranges. Scaling makes Gradient Descent easier to optimize and makes the learning rate more practical.

3. Section B — Normal Equation

The Normal Equation is the closed-form solution for Linear Regression. It calculates the model parameters directly instead of using iterations.

$$\theta = (X^T X)^{-1} X^T y$$

How it works: X contains the bias column and standardized features. Calculate $X^T X$, its inverse, $X^T y$, and multiply them to obtain θ . Predictions are then made using:

$$\hat{y} = X\theta$$

Important: there is no learning rate and no iteration loop in the Normal Equation.

4. Section C — Batch Gradient Descent

Gradient Descent finds the parameters iteratively. At every iteration, it calculates the current prediction error, calculates the gradient, and updates θ .

$$\text{Loss: } J(\theta) = (1 / 2n) \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\text{Gradient: } \nabla J(\theta) = (1 / n) X^T (X\theta - y)$$

$$\text{Update: } \theta \leftarrow \theta - \alpha \nabla J(\theta)$$

- θ : model parameters.
- α : learning rate; controls how large each update is.
- **Iterations:** number of parameter updates.
- **Loss history:** stores the loss at every iteration for plotting.
- **Early stopping:** optionally stop when the change in loss becomes smaller than a chosen tolerance.

A good run should show the loss decreasing over iterations. A learning rate that is too small can converge slowly, while one that is too large can make training unstable.

5. Section E — Scikit-learn Comparison

Train `sklearn.linear_model.LinearRegression()` on the same training data and compare its test results with the Normal Equation and Gradient Descent implementations.

Intercept: sklearn handles the intercept automatically, so use the scaled features without manually adding the column of ones.

The sklearn and Normal Equation results should be very close because they solve the same linear regression problem. Gradient Descent should also become close when it converges.

6. Section F — Visualizations

The assignment asks for clear, labeled figures and requires loss vs iterations, validation loss, and a gradient/loss-surface visualization. It also asks for learning-rate and scaled-vs-unscaled comparisons.

Visualization	Purpose
Loss vs iterations	Shows whether Gradient Descent is converging.
Different learning rates	Shows how the learning rate affects convergence.
Scaled vs unscaled	Shows why feature scaling matters for Gradient Descent.
Validation loss	Shows performance on unseen test data.
Loss surface	Shows how loss changes when selected parameters change.

7. Section G — Evaluation Metrics

Evaluate all three implementations on the test set and compare their results.

- **MSE:** Mean Squared Error. Average squared prediction error. Lower is better.
- **RMSE:** Root Mean Squared Error. Square root of MSE and expressed in the same units as the target. Lower is better.
- **R²:** Coefficient of Determination. Indicates how much variation in the target is explained by the model. Higher is better.
- **MAE:** Mean Absolute Error. Optional metric. Lower is better.

8. Final Comparison

Create one table containing the test-set MSE, RMSE, and R² for:

- Normal Equation
- Gradient Descent
- Scikit-learn LinearRegression

9. Final Assignment Checklist

- Load train.csv and test.csv.
- Separate X and y.
- Standardize features using training statistics.
- Add the bias column for the NumPy models.
- Implement the Normal Equation.
- Implement Batch Gradient Descent.
- Support learning rate, iterations, and optional early stopping.
- Store Gradient Descent loss at each iteration.
- Train sklearn LinearRegression.
- Create the required visualizations and save them.
- Calculate MSE, RMSE, R², and optionally MAE.
- Compare all three implementations.

Quick Memory Map

Preprocess → **Standardize** → **Add bias** → **Train** → **Predict** → **Measure** → **Visualize** → **Compare**